Testing AGENTS.md with Claude Code

KJ Monahan

2026-03-12

Overview

Claude Code and similar AI tools, allow for the creation of an overview file in a repo that is read at the start of every conversation. I tested various approaches to an AGENTS.md file and benchmarked the performance of Claude Code on a Drupal site, executing a front-end (FED), back-end (BED), and integration (INT) task similar to common tasks the Forum One development team does on sites we build.

I found that the AGENTS.md files were less effective overall than might be expected. There was no statistically significant difference between the benchmark performance across tests, though there was some in token usage.

Overall, accounting for both token usage and benchmark performance, the AGENTS.md version that used a minified table-of-contents for supporting documentation performed the best. However, given the limited impact on performance of any approach, omitting the AGENTS.md altogether may be a better choice.

Methodology

To simulate a realistic project (but not actually run tests on a client site), I created a new Drupal site using our `drupal-project` starter with a new theme using Gesso. In keeping with Forum One front-end tradition, I used the "Very Scary Celery" project for the designs. I set up the fonts and color tokens and created the header and footer in Storybook, mirroring work that is usually done upfront by the front-end tech lead. I also did a small amount of initial configuration, creating a few paragraph types and adding fields to the Basic page content type.

Why Drupal?

I tested on a Drupal site specifically for several reasons. I wanted to see how the AGENTS.md would perform with a code base that was likely less-represented in training data than Next.js and React. Practically, Forum One also builds and maintains more Drupal sites than we do React sites, so Drupal results would be more immediately relevant. Finally, because Forum One has been working with Drupal for a long time, we have found particular approaches and conventions that represent our best practices and are bundled into our starter kits. I wanted to see how AGENTS.md would perform not just on a generic Drupal or React repo but specifically with our tools.

Tasks

I wrote prompts for three tasks. The FED task was to create the film detail hero component, using a screenshot of the design. The INT task was to integrate the footer, including creating a custom block for the contact information, using the `footer.twig` and related files I had already styled. The BED task was to make the title field of the Hero paragraph required if "Is Homepage?", a custom field, was checked on the Basic page, and to then use the Hero paragraph title in place of the node title when displaying the page.

Evaluation and Metrics

Using Claude Code, I generated Python scripts used to automate testing of the results. I supplemented this with manual testing, particularly for the INT task, which could be done in ways that failed the automated

tests but I considered acceptable.

Most importantly, the benchmarks I used were intentionally subjective and focused on whether the task was done "right." To be useful, it's not enough for Claude Code to complete the task. It needs to complete the task in a way that maintains quality. The benchmarks therefore were intended to represent the kinds of checks a tech lead might make during a code review. They are by no means exhaustive (and, while testing, I found that Claude Code sometimes hit on solutions that met most of the criteria but were overly complex in other ways) but provide a snapshot of how well Claude Code understood the code base it was working within.

Front-End

- SCSS file is created as a module and so filename does not start with `_`
- A Drupal library entry is created for the new component and the library attached to the Twig file
- All CSS class names start with `c-`, the namespace we use for components
- CSS logical properties are used in place of directional properties (i.e. `inset-block-end` instead of `bottom`)
- Design token accessor functions are used, at a minimum, for fonts and/or spacing
- All hard-coded dimensions use our `rem()` Sass conversion function, not `px`
- The Storybook story was created using a YAML file for the mock data and a `.jsx` file that imports the Twig and YAML file

Integration

- Configuration for a new Block Content type added and used for the Contact Us block. Creating a new PHP block or not creating a new block type at all were both failures
- Existing `menu-social.twig` file is included or embedded in a Drupal Twig template
- The `footer.twig` file already created had three slots for content. Correct placement is:
  - Contact Us block in first slot
  - Footer menu and social menu in second slot
  - Copyright in third slot
- Copyright year is dynamic, not hard-coded to 2026 (or any other year)
- The existing `footer.twig` file is included or embedded in a Drupal Twig template

Manual checking was often needed for the placement, as in some tests, Claude Code would mirror the page wrapper in Storybook by embedding the `footer.twig` in `page.html.twig`, which the automated tests did not catch. However, I judged this to be an acceptable solution (even potentially preferable to modifying `region--footer.html.twig`, since it more closely followed how the `footer.twig` file is used in Storybook).

Back-End

- `hook_field_widget_complete_paragraphs_form_alter` used to implement the conditional required field
- `hook_preprocess_node` used to implement the page title override. Using `hook_preprocess_page_title` was a failure because the `node--full.html.twig` renders the page title
- Hooks are not placed in the `.theme` file. They are either in a module or in one of the `.inc` files Gesso uses to organize hooks
- Hooks follow Drupal conventions of having a `Implements hook_` docblock comment

Across all tests, no implementation ever scored 100% on the back-end benchmarks, but at no point did Claude Code make use of the correct hook. It generally tried to handle the task through various page validation overrides, sometimes with custom JavaScript added as well. Further experimentation is needed to determine if performance could be improved with a better prompt or other nudge.

Testing Process

I used Claude Code to generate bash scripts that created 10 copies of the same test repo using git worktrees, modified the `ddev` config to give each one a unique name, started the sites, installed Composer dependencies,

installed theme dependencies and built the theme, and then performed a site install using the existing config.

Another bash script passed the same prompt to all 10 sites in parallel and created a log file with the session IDs. Once all tests reported finished, I ran the review script for the task, manually checking as well as needed. Then I used a previously created script to parse the `.jsonl` files that contain the transcripts of the sessions (using the session ID from the log file) to calculate the total number of tokens used.

I did not distinguish cache versus uncached tokens or read versus write tokens. I used the total, including the total from any subagents launched in that session.

All tests used Claude Code v2.1.69 with model Sonnet 4.6.

AGENTS.md Formats Tested

I ran all tests for six different iterations of the AGENTS.md file. In between each iteration, the test sites were torn down, any previous documentation removed, new files committed, and the test sites re-spun-up.

No AGENTS.md No AI-specific documentation added. Any existing documentation written for a human audience (such as the READMEs) remained in place.

Claude-generated I ran Claude Code's `/init` command on the base repo and used the resulting file. I did not make any changes other than to rename the file to AGENTS.md, symlinking it to CLAUDE.md and GEMINI.md, so that it would be consistent with future tests.

GenAI v1 I followed the set-up steps in the README for the f1-genai-starter. This uses a central AGENTS.md file with most documentation split into a separate directory and linked from the AGENTS.md file.

GenAI v2 I followed the set-up steps for the not-yet-merged README for the f1-genai-starter. This uses a similar approach as the first, but with files not related to Drupal removed and with a script that creates a minified list of files names and paths to documentation at the bottom of the file.

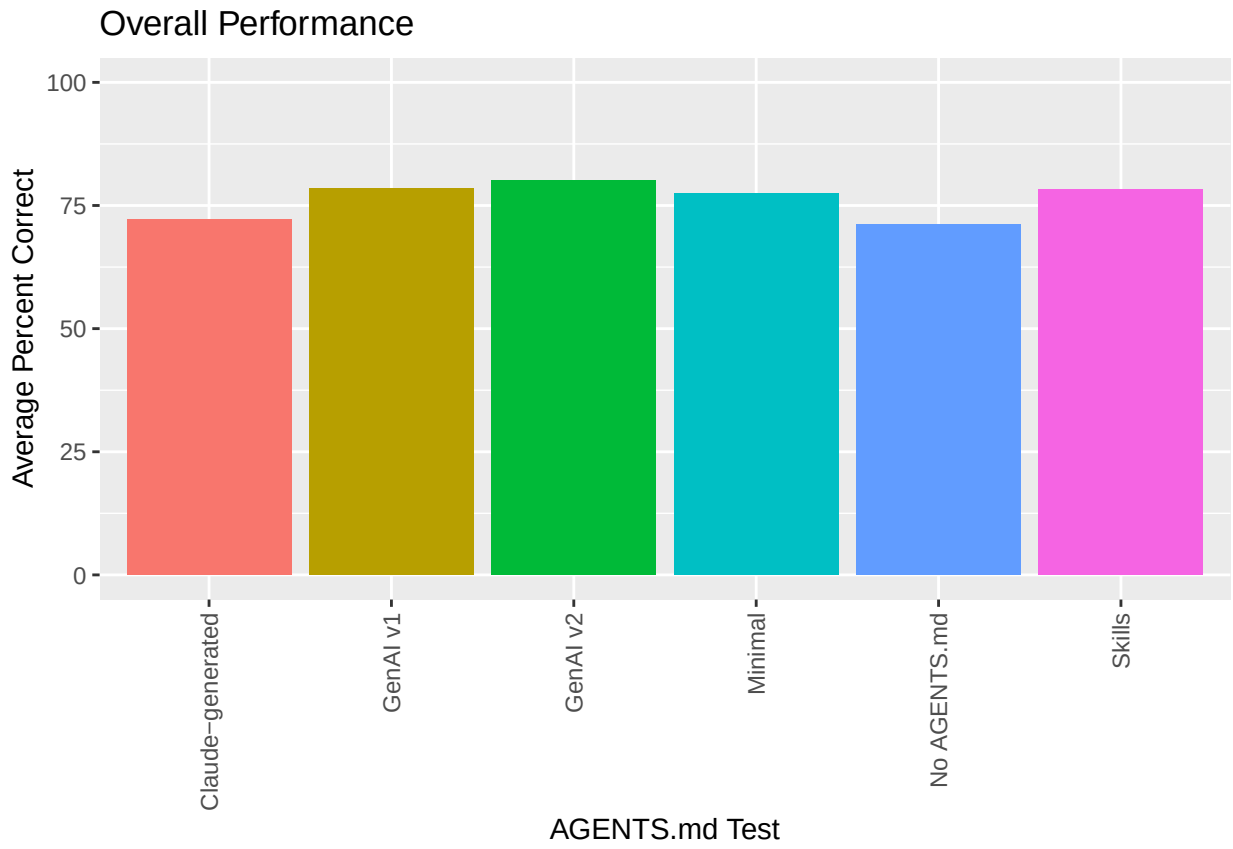
Skills I migrated the contents of `agents_docs` from GenAI v2 to skills. This involved changing the name and location of the Markdown files and adjusting the front-matter. I did not make changes to the files themselves other than to fix references to other documents. I also added a `create_component` skill, which mirrors the one already in f1-genai-starter, but adapted to work in Gesso. There was no AGENTS.md file in this version, only the documentation included in the skills.

Minimal I created a new AGENTS.md file that listed the basics of the tech stack (Drupal, PHP 8.3, ddev), key contrib modules (taking the list from `forumone/canvas`) and pointers to the custom theme and theme helper module.

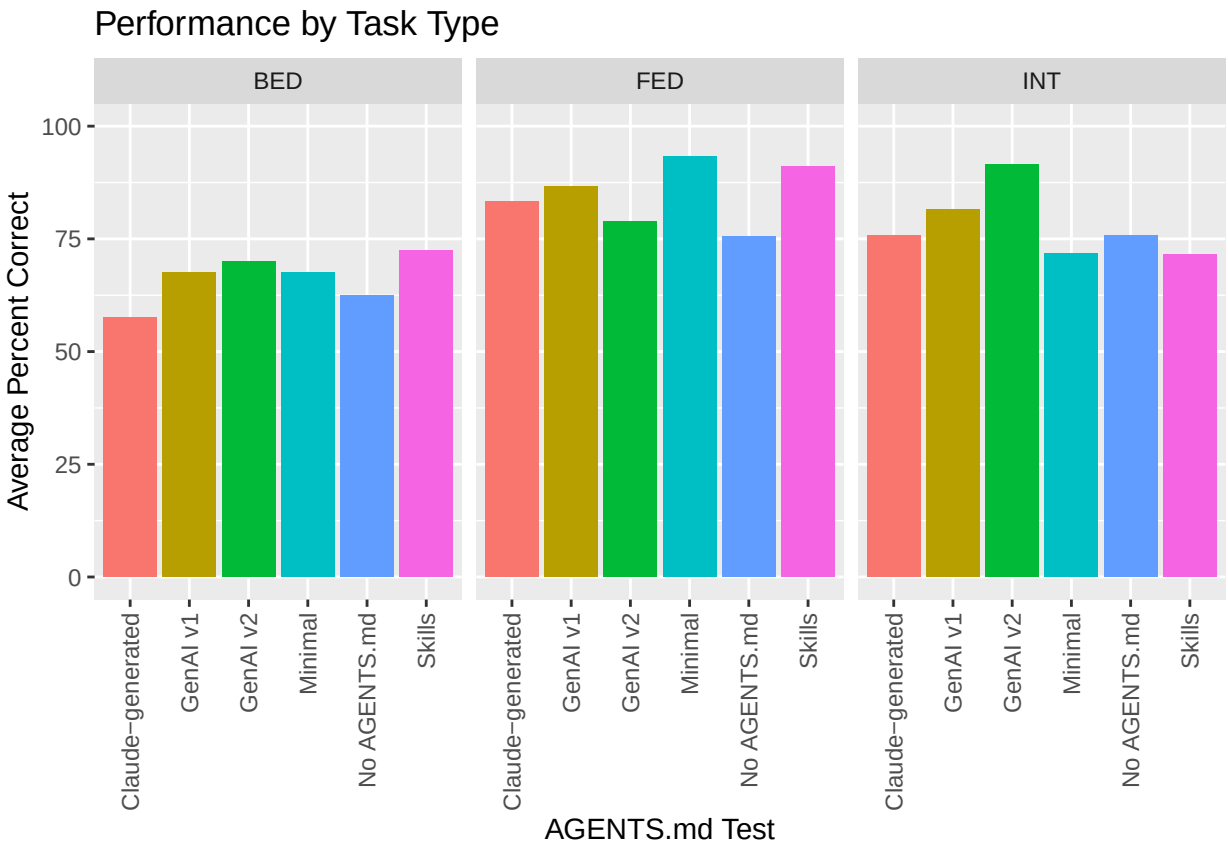
Results

Performance on Benchmarks

Comparing the mean percentage score across all tasks, the GenAI v2 AGENTS.md file did the best, followed by the GenAI v1 approach.

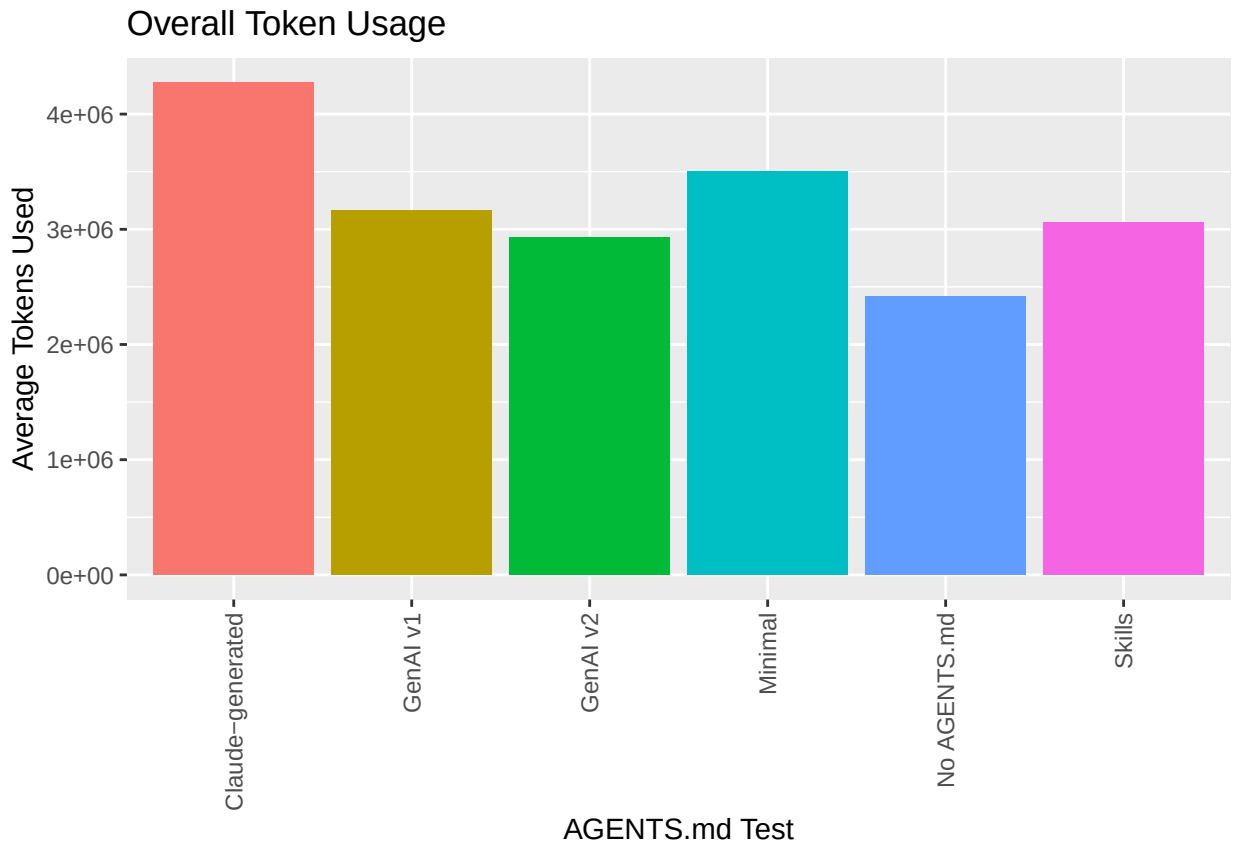


When split by task type, the GenAI v2 approach did particularly well for the INT task. The Skills approach performed well with the FED task, which could be because the `create_component` skill is intended for tasks like that one. However, the Minimal approach performed even better, despite having no information about the front-end other than the location of the theme.

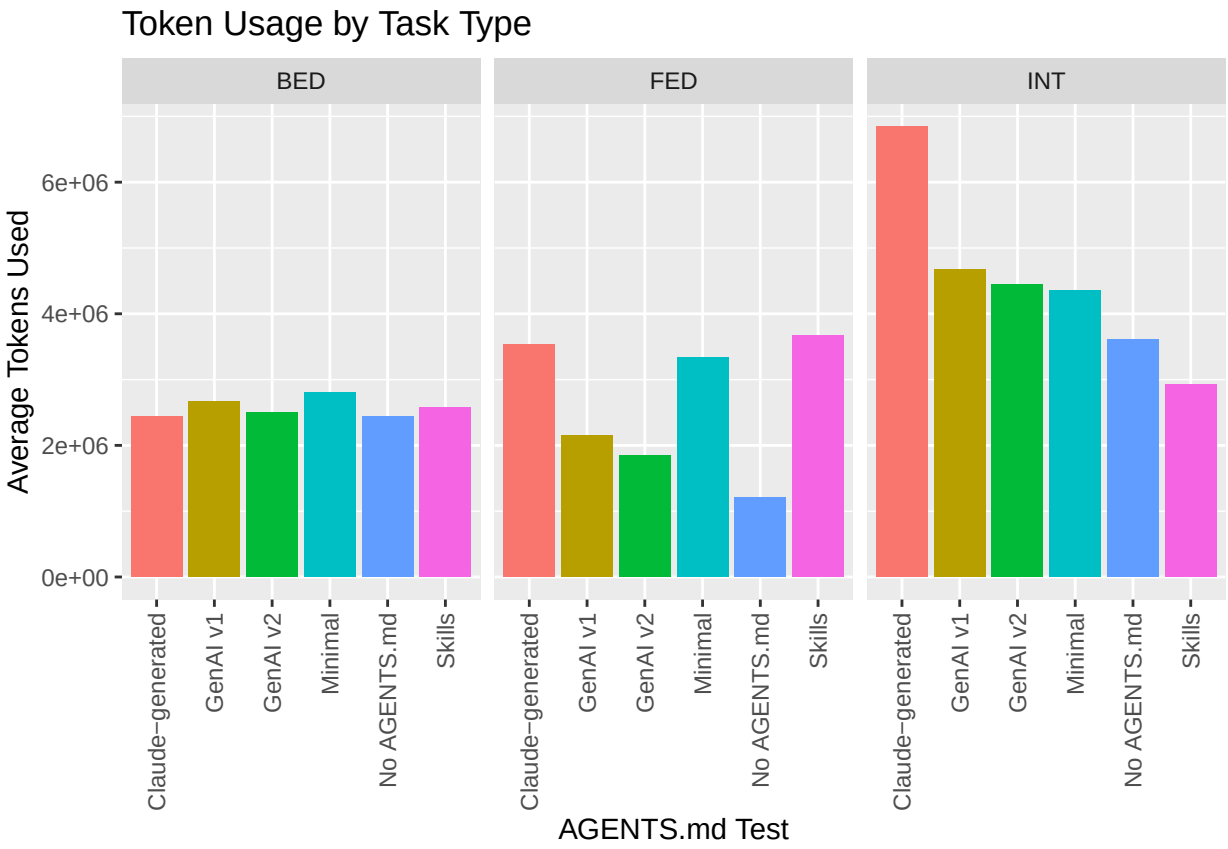


Token Usage

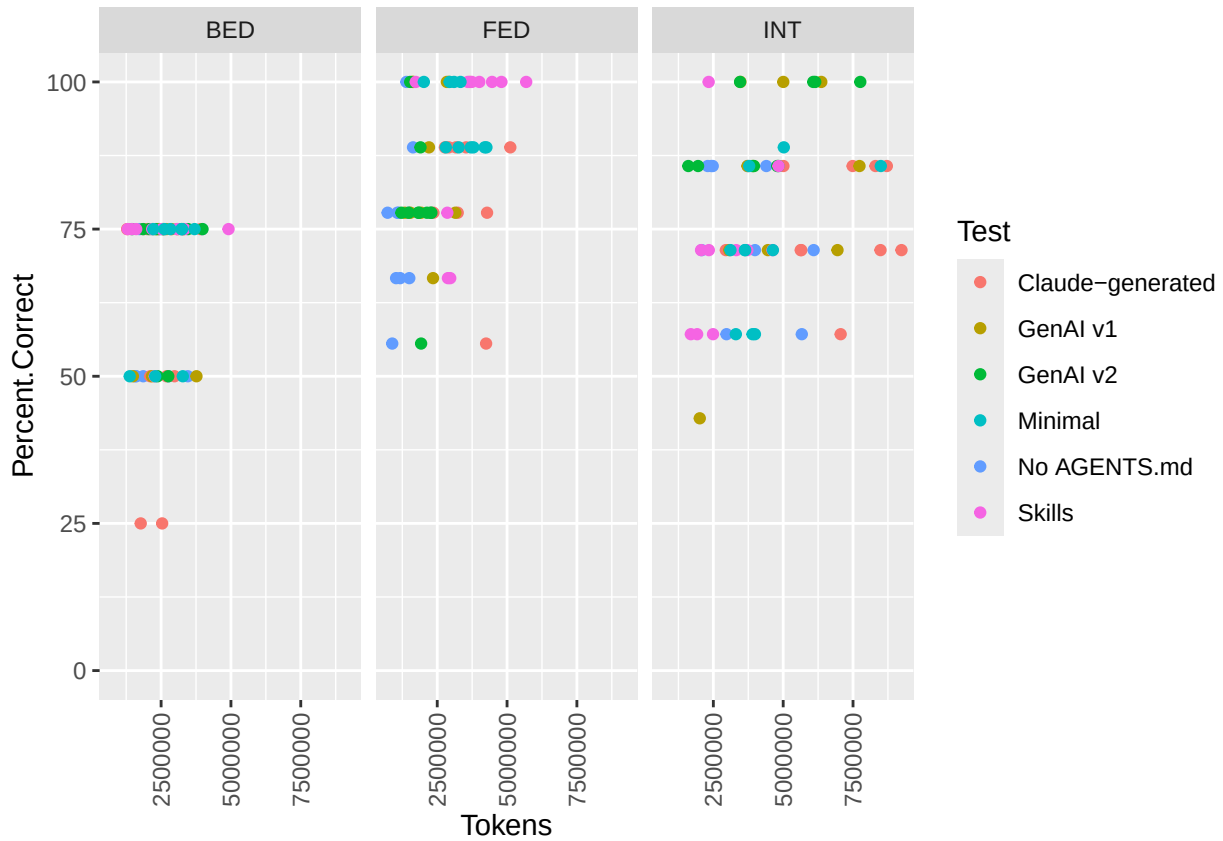
Comparing the mean token usage across all tasks, the Claude-generated AGENTS.md file used the most tokens, and having no AGENTS.md file at all used the fewest. Since the AGENTS.md file is loaded into context for every task, it uses tokens whether or not the information is required for the task.



When split by task, the INT task required the most tokens overall, especially when using the Claude-generated AGENTS.md file. Not having an AGENTS.md file present at all used far fewer tokens but also had the worst relative performance on that task.



However, when looked at across tests and tasks, there is little correlation between token usage and task performance. More tokens does not necessarily result in a better result.

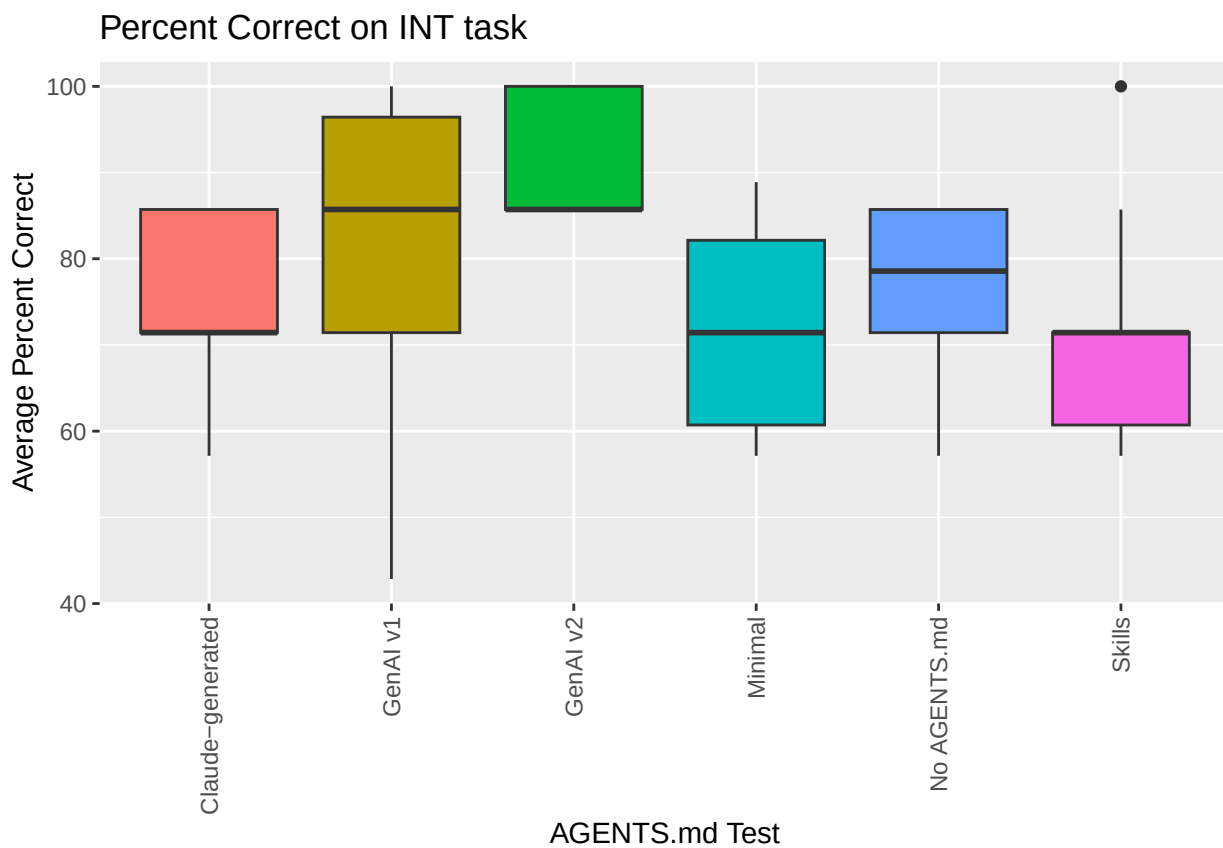


Significance

Looking at the effects of the various AGENTS.md tests on the task performance, none of the differences can be considered statistically significant.

```
##  contrast                  estimate   SE  df t.ratio p.value
##  (Claude-generated) effect    -4.14 2.55 174  -1.625  0.2796
##  GenAI v1 effect               2.21 2.55 174   0.866  0.5145
##  GenAI v2 effect               3.78 2.55 174   1.483  0.2796
##  Minimal effect                1.20 2.55 174   0.471  0.6379
##  No AGENTS.md effect          -5.07 2.55 174  -1.989  0.2796
##  Skills effect                 2.02 2.55 174   0.793  0.5145
##
## P value adjustment: fdr method for 6 tests
```

While there is variability between the AGENTS.md test results, there is also variability in the results between tests on the same task with the same AGENTS.md. That can be seen by looking at the range of results for each test on the INT task.



The token usage comparison does indicate a statistically significant increase in token usage with the Claude-generated file and decrease in token usage with no AGENTS.md file.

```
##  contrast                  estimate     SE  df t.ratio p.value
##  (Claude-generated) effect  1050575 264000 174   3.972  0.0006
##  GenAI v1 effect             -61920 264000 174  -0.234  0.8152
##  GenAI v2 effect            -294779 264000 174  -1.115  0.4520
##  Minimal effect              274174 264000 174   1.037  0.4520
##  No AGENTS.md effect        -805364 264000 174  -3.045  0.0081
##  Skills effect              -162686 264000 174  -0.615  0.6471
##
## P value adjustment: fdr method for 6 tests
```

Combined Metric

To gauge how the tests did when considering both token usage and performance, I centered both columns at zero and scaled them. I then calculated the total, using the inverse of the token usage value. That would result in the highest score being the test with the token usage furtherest below the average and the percent correct highest above the average.

```r
# Overall
grouped |>
  group_by(Test) |>
  summarize(score = mean(score)) |>
  arrange(desc(score))
```

```
## # A tibble: 6 x 2
##   Test                score
```

```
##   <fct>              <dbl>
## 1 GenAI v2          0.603
## 2 Skills            0.327
## 3 GenAI v1          0.267
## 4 No AGENTS.md      0.121
## 5 Minimal          -0.0931
## 6 Claude-generated -1.22
```

```r
# FED
grouped |>
  filter(Task == "FED") |>
  group_by(Test) |>
  summarize(score = mean(score)) |>
  arrange(desc(score))
```

```
## # A tibble: 6 x 2
##   Test             score
##   <fct>            <dbl>
## 1 GenAI v1         1.86
## 2 Minimal          1.60
## 3 No AGENTS.md     1.49
## 4 GenAI v2         1.33
## 5 Skills           1.12
## 6 Claude-generated 0.459
```

```r
# INT
grouped |>
  filter(Task == "INT") |>
  group_by(Test) |>
  summarize(score = mean(score)) |>
  arrange(desc(score))
```

```
## # A tibble: 6 x 2
##   Test              score
##   <fct>             <dbl>
## 1 GenAI v2          0.552
## 2 Skills           -0.259
## 3 No AGENTS.md     -0.361
## 4 GenAI v1         -0.612
## 5 Minimal          -1.33
## 6 Claude-generated -2.87
```

```r
# BED
grouped |>
  filter(Task == "BED") |>
  group_by(Test) |>
  summarize(score = mean(score)) |>
  arrange(desc(score))
```

```
## # A tibble: 6 x 2
##   Test               score
##   <fct>              <dbl>
## 1 Skills            0.119
## 2 GenAI v2         -0.0689
## 3 GenAI v1         -0.443
## 4 Minimal          -0.549
```

```
## 5 No AGENTS.md     -0.766
## 6 Claude-generated -1.26
```

Conclusion

Looking at the overall score, the GenAI v2 approach, with the minified list of file names added by script to the AGENTS.md, had the best combination of token usage and overall performance. However, when split by task, it performed comparatively less well on the FED task and slightly worse than Skills on the BED task. We can also see that there is a wide range of average percent correct within a test, with plenty of overlap, and no approach to AGENTS.md consistently resulted in a higher average percentage correct across all task types.

Thus, it seems just as plausible to conclude that we should optimize for reduced token usage and omit an AGENTS.md file from projects initially, at most adding one only when there is some aspect of the project that Claude Code cannot discover on its own.